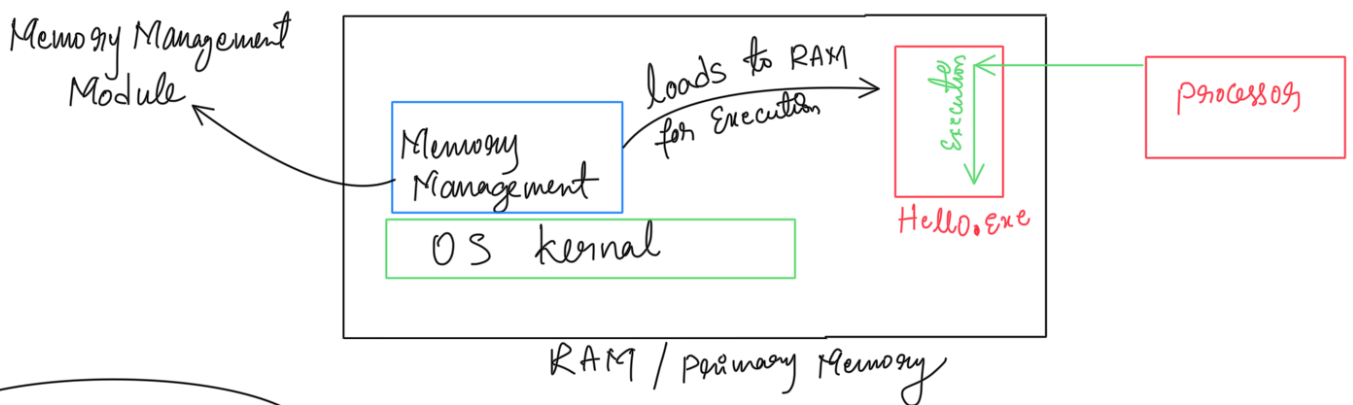


Memory Management

DAY 55
03/09/2025

Memory Management is a core function of an Operating System (OS)
It ensures

- Allocation/deallocation of memory to processes
- Safe and efficient memory usage
- Isolation and protection between processes



At High level

Each process when it is clicked the message was sent to memory manager to load that process to the memory

Memory manager loads the .exe file of the process to the RAM (Primary Memory)

When the process is loaded into RAM and listed in process page table. after that the processor will start executing the process

Memory Manager loads the process to the RAM

There are 2 techniques for loading.

Imp

1) Segmentation → 32 bit old, Embedded

2) Paging → Modern 64 bit, Linux, window, mac

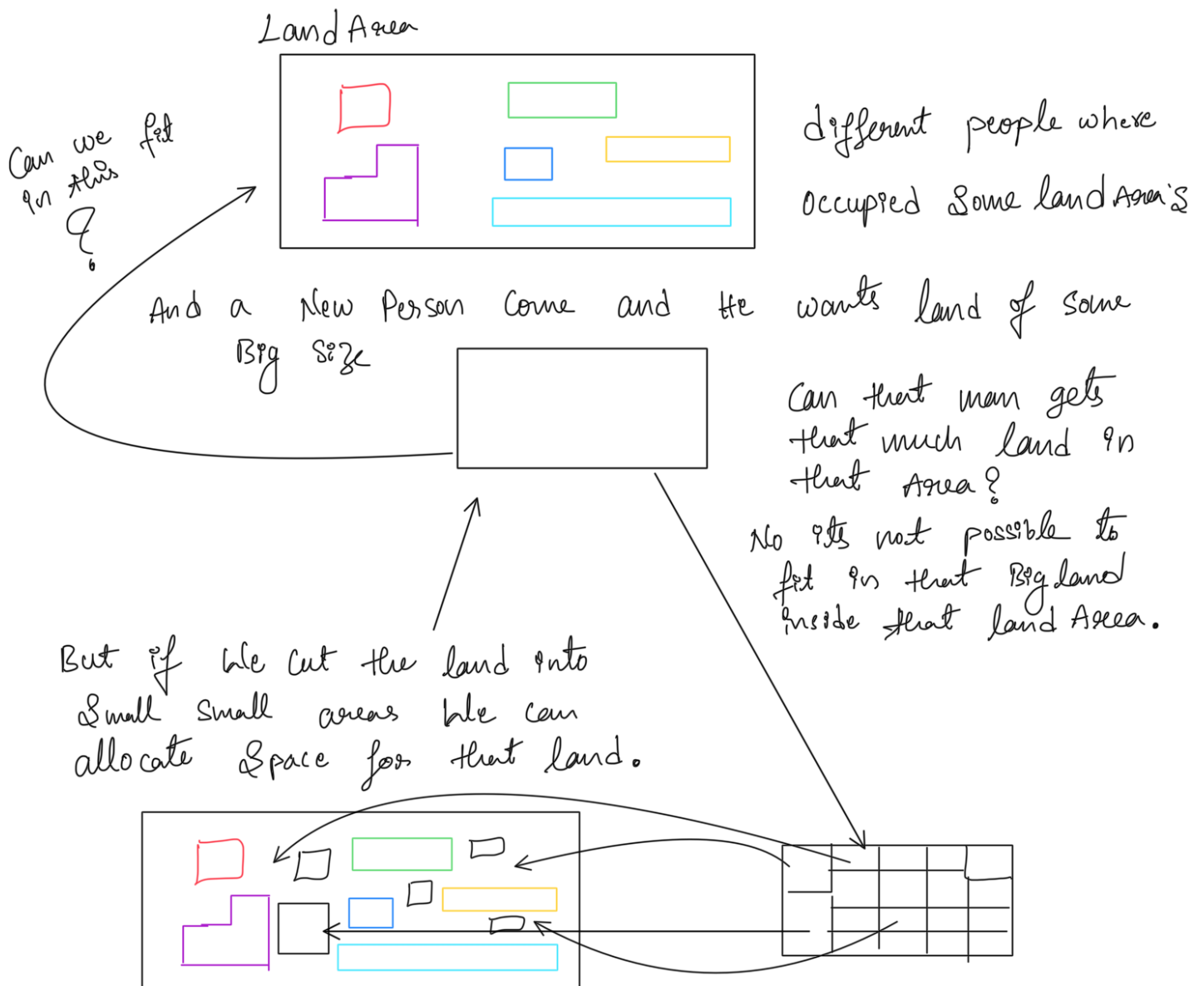
Segmentation, Paging

When we start running a process or Application for that process or Application how Memory is allocated for them in RAM is taken care by the Segmentation and Paging.

Basically Segmentation & Paging are the techniques used to load or allocate Space in the RAM for running a process or Application.

Why this techniques evolved and why this exists?

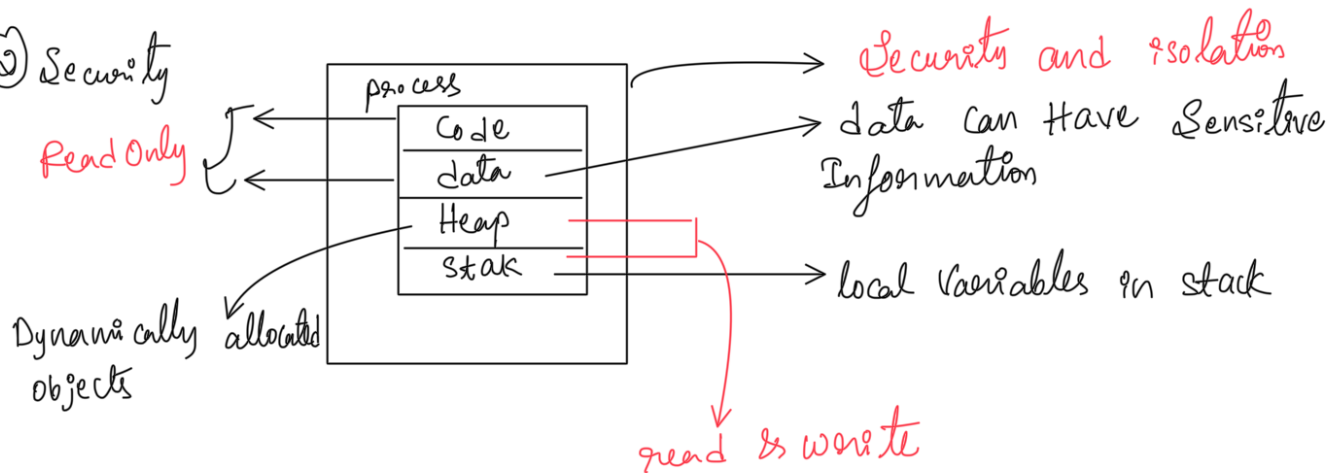
(1) Suppose let us understand with an Example



Now we can allocate the land for it By Dividing large area into Small Areas and providing Space for that Small areas in the Memory.

This technique is called fragmentation
The Big Memory requirement is full filled By creating fragments of it and allocating Space for those fragments

② Security

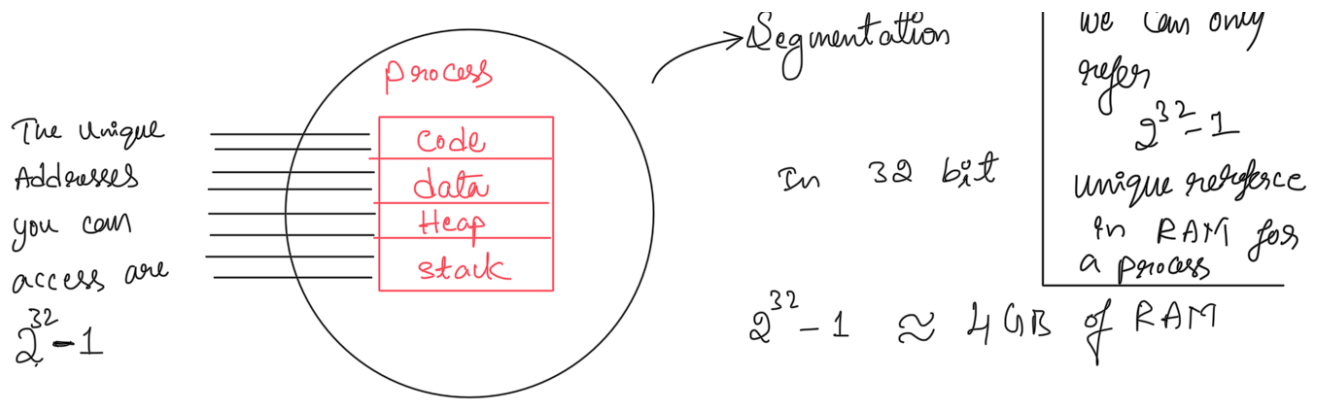


The Code & data are Read only the data cannot be modified
But the stack & Heap data can be modified

This was done for Security Reason this protects the data.

When Initially Operating Systems or Embedded systems are designed at that time only the concept of Segmentation came.

So for Security & Isolation purpose Every process is divided into 4 blocks Code data stack and Heap.



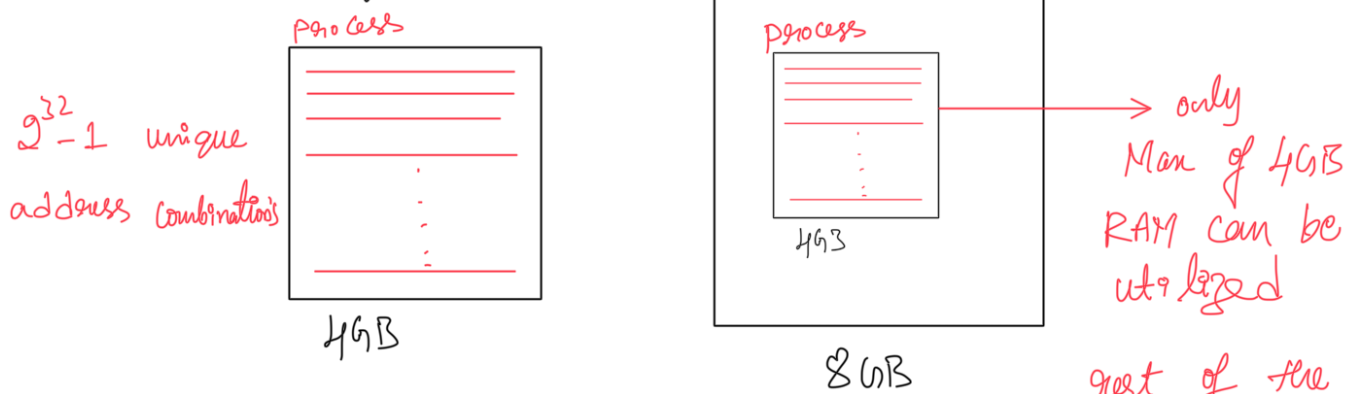
32 bit means $\rightarrow$ the address or references of variables objects $\rightarrow$ have the address in 32 bit

So with 32 bit address there can be $2^{32}-1$ unique addresses are created in RAM. So this creates a disadvantage.

The disadvantage is that as we know the address is 32 bit we can only form $2^{32}-1$ addresses which is approximately of 4GB of space can be covered.

If computers have RAM of 8GB 16GB any extra capacity RAM but computers have potential of using

only 4GB of RAM maximum, Bcz of the address size is 32 bit the maximum combination of unique addresses are formed can only cover upto 4GB of RAM.



rest of the RAM remains unused.

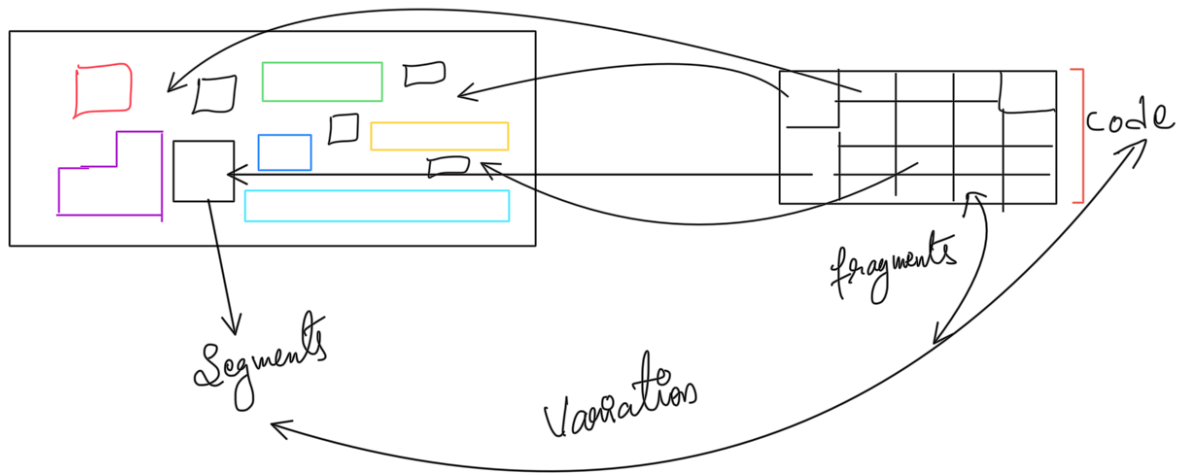
So per process maximum memory

~ Which is allocated is 4GB Bcz the address size is 32bit
1 process can take max space of 4GB in the RAM

fragmentation

fragmentation is another disadvantage of fragmentation.

due to code & data segment variation



Bigger the Source code ↑

Bigger the code Segment ↑

Source code size $\propto$ code segment size

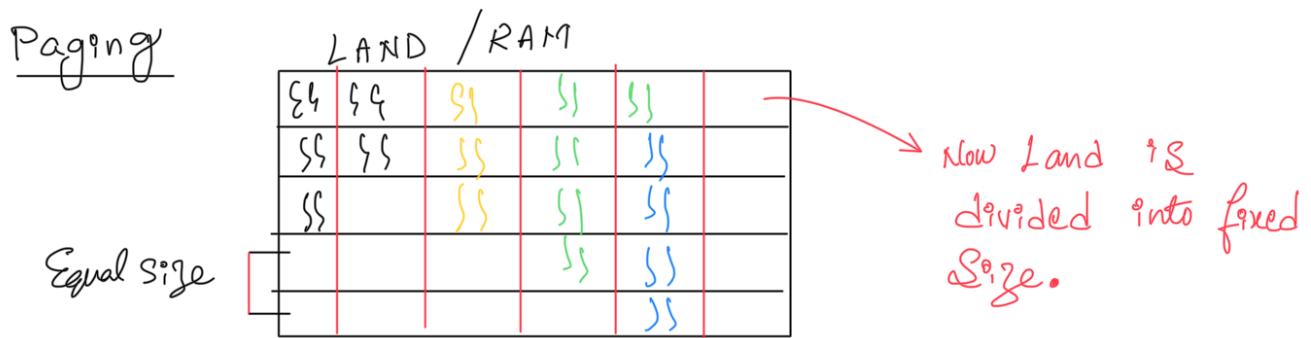
③ Segmentation causes some security issues while executing any program in the kernel mode.

Bcz the kernel mode is a very powerful mode to run a program. If in case of any malicious code in the fragment then it can be executed

In the kernel mode which causes a security issue.

So In Modern Operating System the usage of Segmentation technique has been stopped.

So new technique is introduced its called Paging



The code always deal with blocks (Equal blocks)

The process can take the blocks of code in units multiple of already fixed size of blocks.

We use paging technique to make smaller parts to minimize the wasting of space

Compared to fragmentation

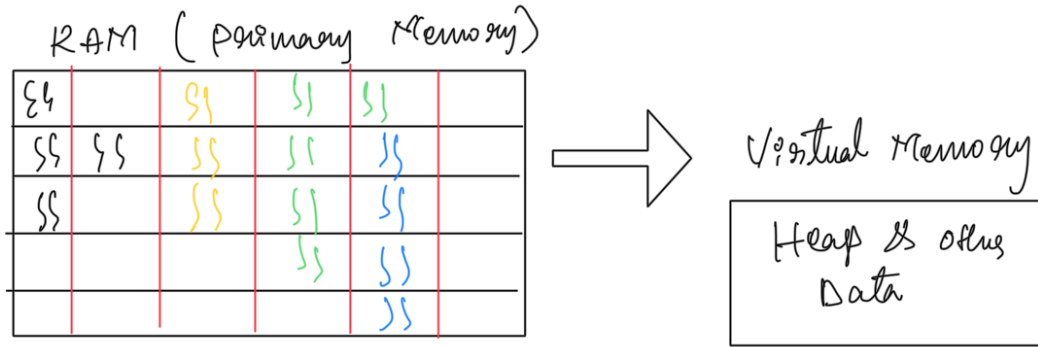
only data & stack get memory allocated in the RAM

Heap and other data are allocated in Virtual Memory

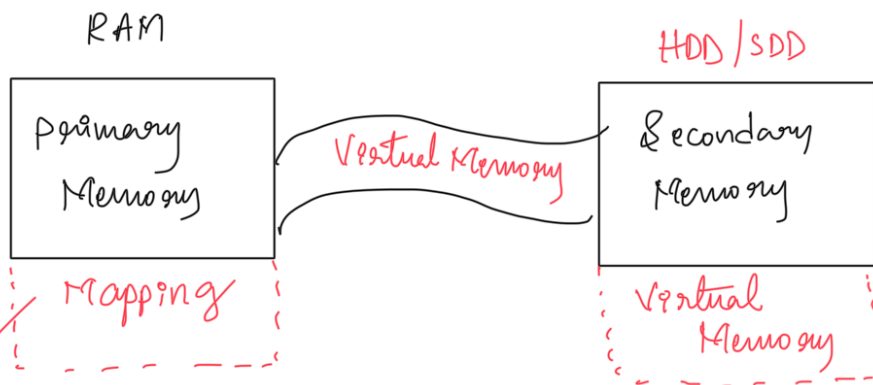
only The code segment & memory required to run the program is allocated in the RAM by allocating respective Pages.

Heap Memory and Some part of the code are not loaded

into RAM, they are allocated space in Virtual Memory



Virtual Memory



The Virtual Memory is stored is stored physically in the Secondary Memory.

Conceptually the Virtual Memory lies in between the primary and Secondary Memory

primary Memory only stores Mapping or address

Problems Before Virtual Memory

In early computer systems (before 1960s):

- Physical Memory was very small** (say a few KBs or MBs).
 - Programs were often bigger than the available RAM.
 - Only small programs could run at once.
- Programmers had to manage memory manually.**
 - They needed to know exactly where instructions/data were stored in RAM.
 - This made coding very complex.
- Fragmentation Issues.**
 - Fixed partitioning wasted memory.
 - Variable partitioning caused external fragmentation.
- No Multiprogramming.**
 - Only one program could run at a time if memory was full.
 - This led to poor CPU utilization (CPU sat idle when waiting for I/O).

Virtual Memory was introduced to overcome these limitations:

- Run programs larger than RAM**
 - Programs can be partially loaded into RAM.
 - Rest of the program stays in disk (secondary storage).
 - Only the "needed" parts (pages/segments) are brought into RAM when required (demand paging).
- Give each process an illusion of large, continuous memory**
 - Even if RAM is small (say 4 GB), OS can use disk to give each process a "virtual address space" (say 16 GB).
 - Each program feels like it has full memory to itself.
- Simplify programming**
 - Programmers no longer worry about physical addresses.
 - They work with virtual addresses, and OS + hardware (MMU) translate them into physical addresses.
- Enable Multiprogramming**
 - Multiple programs can reside in memory simultaneously.
 - Improves CPU utilization and system throughput.
- Isolation & Protection**
 - Each process runs in its own virtual address space.
 - Prevents one process from accidentally (or maliciously) accessing another's memory.

How It Works

- OS divides memory into **pages** (fixed-size blocks).
- When a process accesses a page not in RAM → **Page Fault** occurs → OS brings it from disk to RAM.
- **Page Table** keeps track of mappings between virtual and physical addresses.
- Hardware unit called **MMU (Memory Management Unit)** handles fast address translation.

Real-Life Analogy

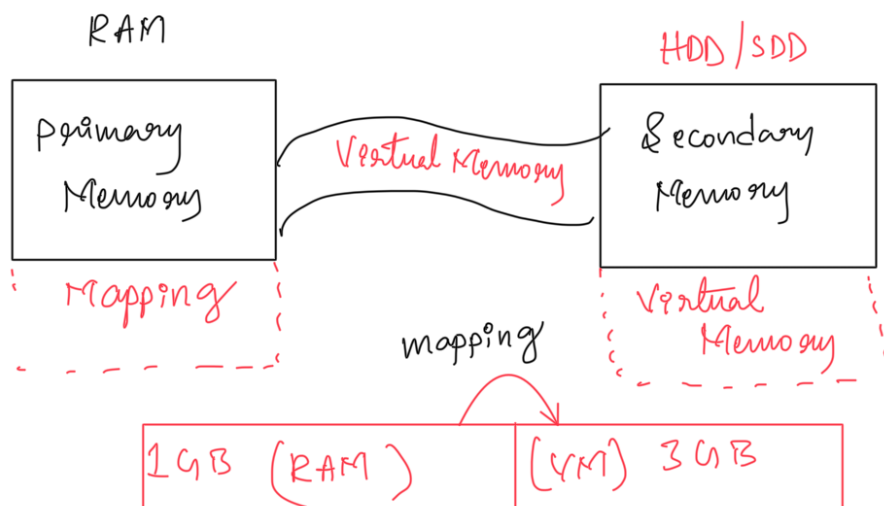
Imagine you are reading a **big book** 📖, but your desk is small.

- You can't keep the whole book on your desk (like RAM).
- You keep the full book in the cupboard (disk).
- You only keep the current chapter (pages) you are reading on the desk.
- If you need another chapter, you replace the old one.

This is Virtual Memory in action.

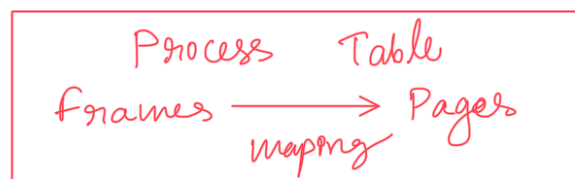
The concept of Virtual Memory came into the picture.

- Overcome limited physical memory
- Simplify programming
- Support multiprogramming
- Provide memory protection
- Efficiently use CPU and memory.



frames → pages

Process table have mapping between frames and pages



Definition of Virtual Memory

Virtual Memory is a memory management technique in Operating Systems that gives each process the illusion of having a large, continuous block of

Virtual memory is a memory management technique in operating systems that gives each process the illusion of having a large, continuous block of memory, even though the computer may have limited physical RAM.

It is implemented by using a combination of **RAM + secondary storage (disk/SSD)**, with help from the **Memory Management Unit (MMU)**.

◆ Key Features of Virtual Memory

1. **Illusion of Large Memory**
 - Even if a system has 4 GB RAM, the OS can create a 16 GB *virtual address space* for each process.
2. **On-Demand Loading**
 - Programs are not fully loaded into RAM.
 - Only the required parts (**pages/segments**) are loaded, the rest stays on disk.
3. **Memory Protection**
 - Each process gets its own virtual address space.
 - Prevents one process from interfering with another.
4. **Multiprogramming Support**
 - Many programs can run at the same time, even if total size > RAM.
5. **Efficient CPU Utilization**
 - CPU can continue running different processes while OS handles memory transfers.

◆ How It Works (in short)

- Memory is divided into **pages** (fixed blocks).
- OS maintains a **page table** mapping virtual addresses → physical addresses.
- If a program accesses a page that's not in RAM, a **page fault** occurs.
- OS brings the required page from disk into RAM.

◆ Real-Life Analogy

Imagine you're working on a **big book** 📖 at a small desk:

- The whole book can't fit on the desk (RAM).
- You keep the book in a cupboard (disk).
- You place only the current chapter (pages) you need on the desk.
- If you need another chapter, you replace the old one.

👉 That's exactly how virtual memory works.

◆ In One Line

Virtual memory = illusion of infinite memory, created by combining limited RAM with disk space, managed by OS + hardware.

◆ Steps in a Page Fault

1. CPU issues memory access (to a virtual address).
2. MMU (Memory Management Unit) checks page table.
3. If the page is not in RAM → **page fault** is triggered.
4. OS handles the fault:
 - Pauses the program.
 - Finds the required page on disk (secondary storage).
 - Loads it into RAM (sometimes replacing an existing page using a **page replacement algorithm**).
 - Updates the page table.
5. Program resumes as if nothing happened.

◆ Types of Page Faults

1. **Minor (Soft) Page Fault**
 - The page is already in memory but not mapped correctly (e.g., just needs to update the page table).
 - Quick to fix.
2. **Major (Hard) Page Fault**
 - The page is not in RAM and must be fetched from disk.
 - Very slow because disk/SSD access is much slower than RAM.

◆ Real-Life Analogy

Imagine you're reading a book 📖:

- You look at page 42 (virtual memory).
- If it's already open on your desk (RAM) → fine.
- If not, you must go to the cupboard (disk), fetch page 42, and place it on the desk.
- That delay = **page fault**.

◆ Why Page Faults Matter

- Normal: Page faults are part of how virtual memory works.
- Too many **page faults** → system slows down drastically (called **thrashing**).

✅ In one line:

A page fault happens when the CPU asks for a memory page that isn't in RAM, forcing the OS to fetch it from disk.

◆ What is a Page Fault?

A page fault occurs when a program tries to access a part of memory (a *page*) that is in its virtual address space, but that page is not currently in physical RAM.

👉 In short: The CPU looks for data in RAM → not found → OS must fetch it from disk.

Higher the page fault slows the application

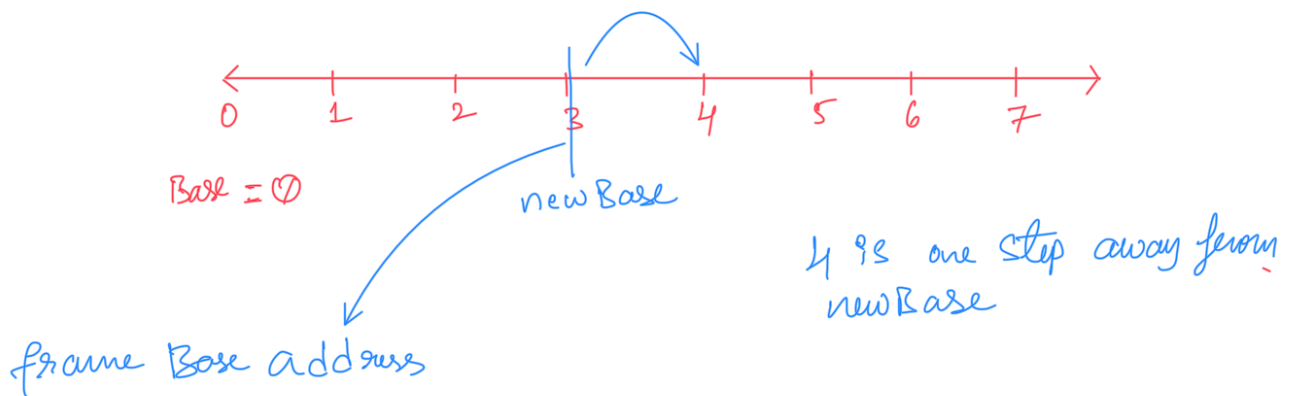
Higher the page fault Higher is the rate of swapping of data from Hard disk to RAM, RAM to Hard disk.

🧠 How Paging Works:

1. Logical address = Page number + Offset
2. Page Table maps pages → frames
3. Translates to physical address

📁 Page Replacement Algorithms

Algorithm	Description
FIFO	Replace oldest loaded page
LRU	Replace least recently accessed page
Optimal	Replace page that won't be used for longest time (ideal)
Clock	Circular queue with reference bits (2nd chance)



RAM (Primary Memory)

SS		SS	SS	SS	
SS	SS	SS	SS	SS	
SS		SS	SS	SS	
			SS	SS	
				SS	

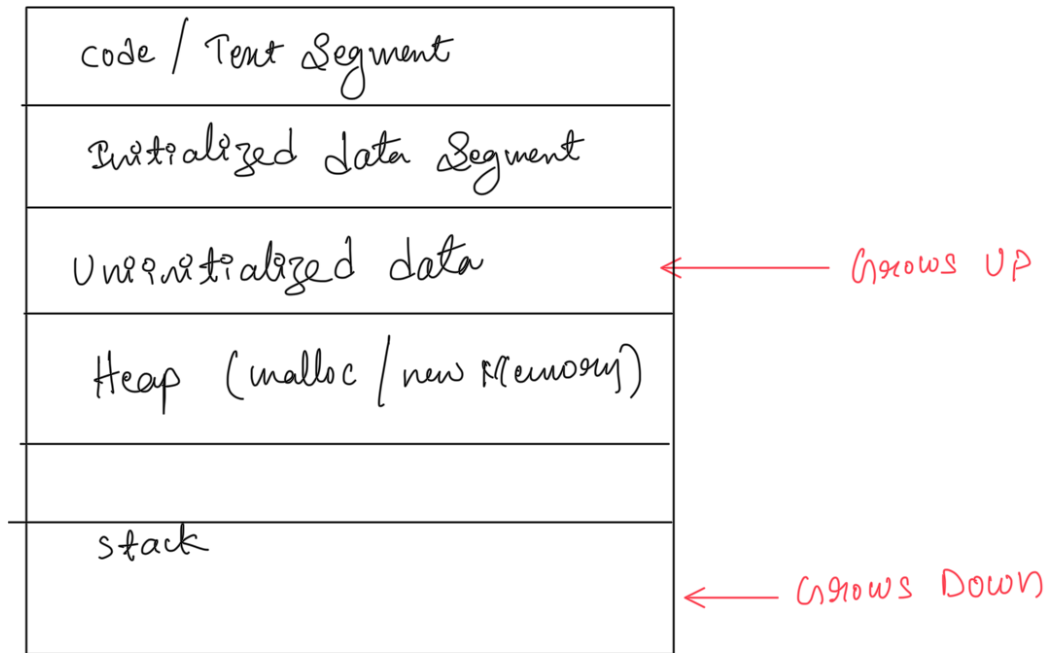
Base address = 1000
of frame

Actual address

$$1000 + 10 = \underline{1010}$$

relative position from frame is called as offset
offset address = 100

Memory Layout With Paging



🐛 Memory Layout in Virtual Address Space

Segment	Allocated at Load?	Grows?	Access
Code	✓ Yes	✗ No	Read + Execute
Data	✓ Yes	✗ No	Read + Write
Heap	✗ On Demand	✓ Up	Read + Write
Stack	✓ Yes	✓ Down	Read + Write

> All segments are composed of pages and mapped via the page table.

Heap Memory used for dynamic memory allocation for a application.

Bcz Load time we don't know How Much Memory is required.

I mean the dynamic Memory allocation is decided At run time.

Operating System Memory Management

Executive Summary

This document explains memory management in operating systems in a structured and intuitive way. It covers key concepts like paging, segmentation, virtual memory, memory layout, page table isolation (PTI), and real-world OS implementations. Analogies and diagrams are used to simplify complex ideas.

What is Memory Management?

Memory Management is a core function of an Operating System (OS). It ensures:

- Allocation/deallocation of memory to processes
- Safe and efficient memory usage
- Isolation and protection between processes

Analogy:

Think of memory as a large apartment building (RAM). The OS is the building manager that rents out apartments (memory blocks) to tenants (processes), ensures no one intrudes into others' apartments, and tracks available rooms.

Types of Memory

1. Primary Memory (RAM)

- Fast, volatile memory
- Holds active processes and data

2. Secondary Memory (HDD/SSD)

- Slower, non-volatile
 - Stores data not in active use (swap, files, etc.)
-

Memory Allocation Techniques

1. Contiguous Memory Allocation

- Each process gets one continuous block of memory
- Can use:
 - **Fixed Partitioning**
 - **Dynamic Partitioning**

2. Non-Contiguous Allocation

- Allocates memory in scattered chunks
- Achieved using:
 - **Paging**
 - **Segmentation**

Paging

What is Paging?

Paging divides memory into equal-size blocks:

- **Physical Memory** → Frames
- **Logical Memory** → Pages

Goals:

- Avoid external fragmentation
- Allow non-contiguous memory use
- Enable virtual memory

How Paging Works:

1. Logical address = Page number + Offset
2. Page Table maps pages → frames

3. Translates to physical address

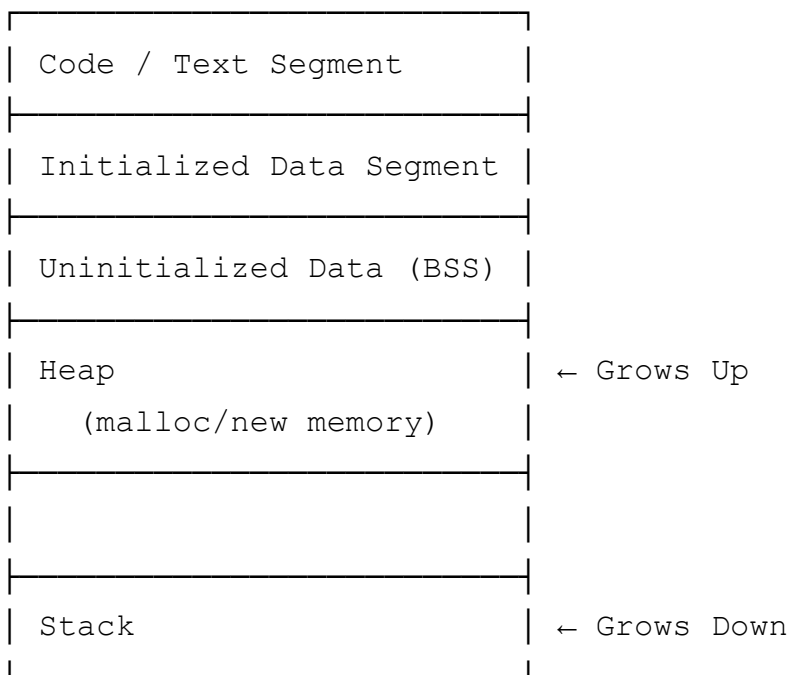
Example Page Table:

Virtual Page	Physical Frame
0	5
1	2
2	8
3	(on disk)

Accessing page 3 → **page fault** → Load from disk

Memory Layout with Paging:

Low Address



High Address

Paging in Windows

- Uses **Demand Paging** + **pagefile.sys**
 - Page size = 4 KB
 - Pages loaded into RAM when needed
-

Page Replacement Algorithms

Algorithm	Description
FIFO	Replace oldest loaded page
LRU	Replace least recently accessed page
Optimal	Replace page that won't be used for longest time (ideal)
Clock	Circular queue with reference bits (2nd chance)

Analogy:

Like cleaning a crowded fridge (RAM) to make space: FIFO removes the oldest item, LRU removes what you haven't touched in days.

Memory Layout in Virtual Address Space

Segment	Allocated at Load?	Grows?	Access
Code	Yes	No	Read + Execute
Data	Yes	No	Read + Write
Heap	On Demand	Up	Read + Write
Stack	Yes	Down	Read + Write

All segments are composed of pages and mapped via the page table.

Page Table Isolation (PTI)

Why PTI?

To protect kernel memory from user access (Meltdown fix).

How PTI Works:

- Separate page tables for user and kernel mode
- Switches on context change

PTI Impact:

Feature	Before PTI	After PTI
Kernel mapped?	Yes	No
Protected by perms?	Yes	Yes
Speculation-safe?	No	Yes

OS Support:

OS	PTI Support
Linux	Kernel 4.15+
Windows	Win 10 v1803+
macOS	macOS 10.13.2+

Segmentation

What is Segmentation?

- Divides logical memory into **logical segments**: code, data, stack
- Each segment has a **number + offset**

Paging vs Segmentation:

Feature	Paging	Segmentation
Division	Fixed-size pages	Logical segments
Fragmentation	Internal	External
Structure	Hardware-enforced	Aligned with program logic

Analogy:

Paging is like cutting paper into squares. Segmentation is like dividing a book into chapters.

OS-Specific Segmentation

Linux:

- Uses **flat segments**
- 64-bit mode disables segmentation except FS/GS
- FS/GS used for TLS (Thread Local Storage)

macOS:

- Based on Darwin (XNU), follows flat model
- Uses paging and virtual memory only

Windows:

- Uses **pure paging**, segmentation obsolete

OS	Segmentation Use?	Notes
Linux	Minimal (FS/GS)	TLS & legacy systems only
macOS	No	Pure paging
Windows	No	Pure paging with flat address

Virtual Memory

What is Virtual Memory?

- Lets processes exceed RAM limits
- Maps virtual pages to disk when RAM is full
- Enables **Demand Paging**

Swapping & Thrashing

- **Swapping**: inactive pages moved to disk

- **Thrashing:** system spends more time swapping than executing
-

Common Interview Questions

1. Paging vs Segmentation

Paging = fixed blocks; Segmentation = logical blocks

2. What is a Page Fault?

Accessing a page not in RAM; must load from disk

3. Virtual Memory?

Use of disk as extension of RAM

4. FIFO vs LRU?

FIFO is simpler; LRU is smarter

5. What is TLB?

Hardware cache for recent page table lookups

6. Internal vs External Fragmentation?

Internal = unused space inside block

External = scattered small free blocks

7. Segmentation Fault?

Illegal access outside segment boundaries

Final Takeaways

- Paging is the dominant memory management method
- Segmentation is mostly historical (except FS/GS)
- Page table isolation is key to security
- OSes like Linux, Windows, and macOS all use pure paging with virtual memory

Modern OSes protect memory via isolation, translation, and permission checks—making multi-process computing secure and efficient.